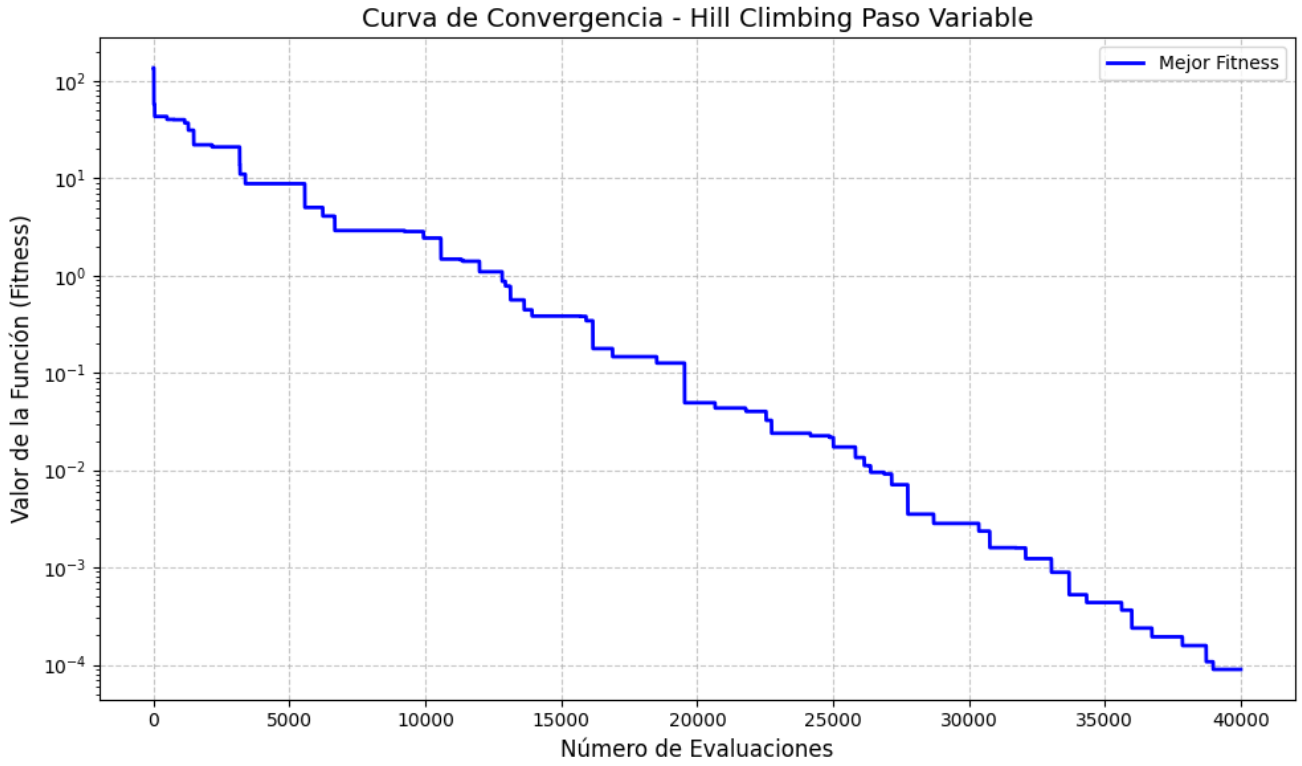
OCTAVO LABORATORIO

En este laboratorio vamos a ampliar el conjunto de funciones a minimizar con 3 funciones más que al igual que la semana pasada se dan de forma ofuscada. En el código proporcionado están también las cuatro de la semana pasada y las tres nuevas que se denominan Funcion_5, Funcion_6 y Funcion_7. Para esta prueba se recomienda que, en vez de fijar la parametrización de los algoritmos de búsqueda durante todas las iteraciones, esta se haga dinámica, de forma que pueda cambiar a lo largo de las evaluaciones. Este mecanismo puede permitir hacer "saltos" más grandes al comienzo y más pequeños al final para ajustar mejor el valor del mínimo.

Como nueva incorporación añada una función que a partir de los datos de cualquier ejecución se pueda observar el avance de la optimización (se ha hecho con 40.000 evaluaciones para una visión más amplia):



Ejercicio voluntario de ampliación.

Si quiere probar un problema de optimización realmente complicado le proporcionamos una nueva función Funcion_8 que es la denominada función de *Schwefel*:

```
import numpy as np
import retos_optimizacion as reto

class Funcion_8(reto._BaseOpt):
  def __init__(self):
    shift_vectors = [0.0] * 10
    super().__init__(shift_vectors)

  def evaluar(self, x):
    z = self._preparar_x(x)
    z = z - self._shift

    # Evaluamos la sumatoria de z * sen(sqrt(|z|))
    sum_term = np.sum(z * np.sin(np.sqrt(np.abs(z))))
    return float(418.9828872724338 * self._dims - sum_term)
```

Para usarla traslade este código a un módulo Python *función_8.py* e impórtelo en el módulo py donde tenga la función de optimización. Esta función tiene un mínimo en x_i=420,9687 para todo i. Esta función debe usarse en los límites [-500,500] para que siempre sea positiva y el mínimo esté en cero.

Puede cambiarse el mínimo de la función modificando el shift_vectors de forma que si lo hace igual a [-419,9687]*10. Entonces el mínimo se desplaza a [1.0]*10. Esta sería la función transformada con el cambio propuesto:

```
class Funcion_8_modificada(reto._BaseOpt):
  def __init__(self):
    shift_vectors = [-419.9687] * 10
    super().__init__(shift_vectors)

  def evaluar(self, x):
    z = self._preparar_x(x)
    z = z - self._shift

    # Evaluamos la sumatoria de z * sen(sqrt(|z|))
    sum_term = np.sum(z * np.sin(np.sqrt(np.abs(z))))
    return float(418.9828872724338 * self._dims - sum_term)
```

Compruebe que en [1.0]*10 se alcanza el mínimo y la función es igual a 0.0

Ahora esta versión de la función de Schwefel es más fácil de optimizar, pero hay que tener en cuenta que se ha hecho “trampas” puesto que para definir el shift_vectors teníamos que saber dónde estaba el mínimo.

Nota: No olvide en todos los ejercicios fijar la semilla de la generación aleatoria a 1